
Training-Free KV Cache Compression and GQA Conversion for Pythia-70M Inference Acceleration

Ruoyu Fu

NLP Course Final Project

Code: github.com/lilfry09

Training-free-LM-inference-acceleration

Abstract

1 Efficient inference is a practical bottleneck for autoregressive language models
2 because attention computation and KV-cache storage grow with context length.
3 This report studies two training-free modifications to Pythia-70M: KVPress cache
4 compression and a reduced-KV grouped-query attention (GQA) conversion. We
5 evaluate chunked teacher-forced perplexity, cached next-token perplexity, TTFT,
6 TPOT, throughput, cache size, a KVPress compression-ratio ablation, and a GQA
7 KV-head sanity check on WikiText-103 and a PG-19 sample in a CPU-only envi-
8 ronment. Overall, this small-scale CPU study suggests that KVPress can help in
9 selected settings, while naive training-free GQA is not a reliable drop-in accelera-
10 tor.

11 1 Introduction

12 Autoregressive decoding repeatedly evaluates a transformer while maintaining a key-value (KV) cache
13 for previous tokens. This cache avoids recomputing attention states, but its memory and attention
14 cost scale with sequence length. Recent efficient-inference work therefore attacks the problem from
15 several directions: using fewer KV heads (Ainslie et al., 2023), more IO-aware attention kernels (Dao
16 et al., 2022), or compressing the KV cache during generation (Xiao et al., 2023; NVIDIA, 2024).

17 This project focuses on modifications that can be reproduced without training a new model. We
18 use Pythia-70M (Biderman et al., 2023), a small GPT-NeoX model, as the common backbone. The
19 assignment asks for both quality and speed evaluation on datasets such as WikiText and PG-19, so we
20 report two complementary quality metrics. Chunked perplexity is computed with teacher forcing on
21 fixed 512-token chunks and is not affected by KVPress, because KVPress acts on generation caches
22 rather than full-context forward passes. Cached perplexity evaluates next-token prediction after a
23 prefill pass whose KV cache may be compressed; this is the relevant quality metric for the KVPress
24 experiments here.

25 The main contribution is a reproducible comparison under a CPU-only course environment. The
26 results show that reduced-KV GQA is not a reliable drop-in conversion for a model trained with
27 standard multi-head attention, while KV cache compression can help on some contexts but is sensitive
28 to runtime overhead.

29 2 Methods

30 **Baseline.** The baseline loads the local EleutherAI/pythia-70m checkpoint through Hugging
31 Face Transformers and uses the default scaled dot-product attention (SDPA) implementation. No
32 weights are changed.

33 **Reduced-KV GQA conversion.** Pythia-70M uses 8 attention heads. In the GQA variant, we
34 compute the original query, key, and value states, average the key/value heads into fewer grouped KV

heads, and update the dynamic cache with those reduced tensors. The main run uses 2 cached KV heads, so each layer stores `[batch, 2, seq, head_dim]` instead of `[batch, 8, seq, head_dim]`. Attention is evaluated with PyTorch SDPA grouped attention, without explicitly materializing an expanded KV cache in our implementation. This is a training-free conversion rather than the fine-tuned GQA recipe of Ainslie et al. (2023), so it is treated as a stress test rather than a competitive GQA recipe.

KVPress cache compression. We evaluate two KVPress modes at compression ratio 0.5. Knorm keeps tokens with larger key-norm scores. StreamingLLM keeps attention-sink tokens and recent tokens, following the observation that early sink tokens are important for stable long-context decoding (Xiao et al., 2023). We use the KVPress compression ratio as the fraction of cached positions removed: ratio 0.5 keeps approximately 50% of the cache, while ratio 0.75 keeps approximately 25%. Compression is applied after prefill via forward hooks on GPT-NeoX attention layers. The code records per-layer cache lengths before and after compression; for example, ratio 0.5 compresses a 512-position cache to 256 positions.

Metrics. All methods use the same token windows. Chunked PPL uses the first 4096 tokens and resets context every 512 tokens: each chunk input predicts the next token inside that chunk, with no KV state carried across chunk boundaries. Cached PPL uses tokens 0–511 as prefill context and scores tokens 512–767 one by one with ground-truth continuation tokens fed back into the cache; KVPress compresses the prefill cache before this scoring step, while baseline and GQA keep their normal caches. KV events count layer-level compression calls across cached-PPL scoring, warm-up, and measured speed prefills. Speed uses the first 128, 512, or 1024 tokens as prompt and manually decodes 64 greedy tokens. Each setting has one warm-up and three measured runs. TTFT is wall time from prefill start to selecting the first token; TPOT is decode time divided by the remaining 63 generated tokens; throughput is 64 generated tokens per end-to-end second. Changes smaller than one standard deviation of the compared throughput measurements are marked as unstable.

3 Experimental setup

We evaluate the first 4096 tokens from the test split of WikiText-103 (Merity et al., 2016). For PG-19 (Rae et al., 2020), we use the first test example returned by Datasets 3.6.0 and tokenize it with the Pythia tokenizer, again taking the first 4096 tokens. The local environment is Windows on an Intel Core Ultra 7 155H CPU with 16 cores, 22 logical processors, 32 GB RAM, PyTorch 2.6.0, Transformers 4.56.2, Datasets 3.6.0, KVPress 0.5.3, and 16 PyTorch CPU threads. CUDA is unavailable, so FlashAttention is not used as a main experimental condition. Greedy decoding is deterministic; no sampling is used. Dependencies are pinned in `requirements.txt`.

The exact reproduction command is:

```
cd finalproj
.\scripts\run_matrix.ps1
```

This script runs baseline, GQA, KVPress-Knorm, and KVPress-StreamingLLM on both datasets and regenerates `comparison_quality.*` and `comparison_speed.*`.

4 Results and discussion

Table 1 summarizes quality. The chunked-PPL column should be used to compare baseline and GQA under fixed chunking. The cached-PPL column should be used to compare KVPress cache-compression behavior, because it scores the same continuation tokens after prefill. GQA causes a large degradation: WikiText chunked PPL rises from 63.72 to 2258.01, and PG-19 chunked PPL rises from 33.57 to 816.70. This confirms that converting a pretrained MHA checkpoint into 2 cached KV heads by simple averaging is not quality-preserving. Mechanistically, averaging key/value heads destroys head-specific representations learned during pretraining; without the uptraining step used in standard GQA conversion, query heads attend to compressed KV states that no longer match their learned attention geometry.

Table 1: Quality metrics. Chunked PPL resets context every 512 tokens over the first 4096 tokens; cached PPL scores tokens 512–767 after a 512-token prefill cache. KVPress does not affect chunked PPL because compression is only applied to generation caches.

Dataset	Method	Chunked PPL	Cached PPL	KV events	Removed frac.
WikiText	Baseline	63.72	13.09	0	–
WikiText	GQA-2KV	2258.01	3659.00	0	–
WikiText	KVPress-Knorm	63.72	52.44	78	0.50
WikiText	KVPress-StreamingLLM	63.72	61.30	78	0.50
PG-19	Baseline	33.57	29.60	0	–
PG-19	GQA-2KV	816.70	776.88	0	–
PG-19	KVPress-Knorm	33.57	31.76	78	0.50
PG-19	KVPress-StreamingLLM	33.57	29.49	78	0.50

The WikiText baseline has much lower cached PPL than chunked PPL. To check whether this was a cache-scoring artifact, we recomputed ordinary teacher-forced PPL on the identical continuation slice: the first 512 tokens were used as context and the next 256 tokens were scored. The same-slice teacher-forced PPL was 13.088, while cached autoregressive PPL was 13.089. This small difference indicates that cached scoring matches standard teacher forcing on the same tokens. The gap from the 63.72 chunked PPL is therefore caused by the measurement protocol: chunked PPL averages over 4095 tokens and resets context every 512 tokens, whereas cached PPL uses one fixed local continuation with an intact 512-token prefill. We therefore compare cached PPL only across methods on the same cached-PPL slice, not against chunked PPL as an absolute model-quality number.

Tables 2 and 3 give the raw speed means and standard deviations used for the relative changes. On WikiText, both KVPress modes improve 128/512-token generation, but slow down at 1024 tokens. On PG-19, StreamingLLM is unstable at 512 tokens and gives a modest +8.7% at 1024 tokens. GQA stores fewer KV heads but is not a useful accelerator because its quality collapses and its speed is inconsistent. Because CPU variance is visible and each setting has only three measured runs, these speed numbers should be read as indicative implementation evidence rather than statistically strong claims.

Table 2: WikiText speed results. Thr. is generated tokens/s. Relative change is against the baseline with the same context. A dagger marks changes smaller than one standard deviation.

Method	Ctx	Thr.	TTFT	TPOT	Rel.
Base	128	50.9±14.6	0.119±0.084	0.0194±0.0058	+0.0%
GQA-2	128	41.4±4.1	0.179±0.060	0.0219±0.0018	-18.7% [†]
Knorm	128	66.8±1.8	0.116±0.014	0.0134±0.0002	+31.3%
Stream	128	68.8±2.4	0.119±0.019	0.0129±0.0002	+35.3%
Base	512	35.0±5.0	0.569±0.090	0.0204±0.0051	+0.0%
GQA-2	512	29.8±6.3	0.448±0.308	0.0281±0.0040	-14.8% [†]
Knorm	512	52.3±1.2	0.333±0.011	0.0141±0.0006	+49.7%
Stream	512	52.5±4.9	0.306±0.014	0.0146±0.0020	+50.3%
Base	1024	51.2±3.4	0.325±0.038	0.0147±0.0007	+0.0%
GQA-2	1024	25.2±4.3	1.084±0.359	0.0239±0.0041	-50.8%
Knorm	1024	43.9±2.4	0.523±0.055	0.0149±0.0022	-14.3%
Stream	1024	42.3±1.9	0.614±0.032	0.0143±0.0009	-17.5%

Table 3: PG-19 speed results. Thr. is generated tokens/s. Relative change is against the baseline with the same context. A dagger marks changes smaller than one standard deviation.

Method	Ctx	Thr.	TTFT	TPOT	Rel.
Base	128	74.3±0.4	0.083±0.003	0.0124±0.0000	+0.0%
GQA-2	128	53.3±5.1	0.138±0.023	0.0170±0.0017	-28.3%
Knorm	128	65.1±4.3	0.128±0.026	0.0136±0.0007	-12.4%
Stream	128	66.4±3.2	0.107±0.005	0.0136±0.0007	-10.6%
Base	512	60.6±2.6	0.236±0.017	0.0130±0.0006	+0.0%
GQA-2	512	19.6±1.3	0.775±0.028	0.0396±0.0039	-67.6%
Knorm	512	51.7±5.0	0.319±0.023	0.0147±0.0017	-14.7%
Stream	512	58.9±4.0	0.279±0.037	0.0129±0.0009	-3.0% [†]
Base	1024	47.3±0.4	0.422±0.005	0.0148±0.0002	+0.0%
GQA-2	1024	18.5±0.5	1.764±0.015	0.0270±0.0015	-61.0%
Knorm	1024	40.7±1.1	0.619±0.026	0.0152±0.0004	-14.0%
Stream	1024	51.4±0.8	0.425±0.015	0.0130±0.0002	+8.7%

Table 4 adds two quality/cache diagnostics. First, setting GQA to 8 KV heads exactly reproduces the baseline PPL, validating numerical correctness of the grouped-attention path for quality metrics; its speed is not used as a baseline replacement. Reducing to 4 or 2 KV heads creates the quality collapse. Second, the StreamingLLM ratio ablation shows the expected cache-size tradeoff: higher removed fraction reduces approximate cache memory but worsens cached PPL. Diagnostic throughput from this separate ablation run is kept in the output files for audit, but the main paper table omits it to avoid mixing absolute speed numbers from separately executed scripts. Approximate cache memory is computed as layers $\times$ KV heads $\times$ cache tokens $\times$ head dimension $\times$ 2 for K/V $\times$ 4 bytes. For Pythia-70M at 512 positions, the baseline cache is about 12 MB; GQA-2KV and StreamingLLM at ratio 0.75 reduce this to about 3 MB.

Table 4: Diagnostic quality/cache ablations on WikiText with a 512-token prompt. Cache MB is an approximate per-sequence KV cache size in float32.

Ablation	Setting	Approx. KV MB	Chunked PPL	Cached PPL
GQA	8 KV heads	12.0	63.72	13.09
GQA	4 KV heads	6.0	1240.55	1976.45
GQA	2 KV heads	3.0	2258.01	3659.00
Stream.	ratio 0.25	9.0	63.72	22.92
Stream.	ratio 0.50	6.0	63.72	61.30
Stream.	ratio 0.75	3.0	63.72	62.61

Threats to validity. The main limitation is scale. All experiments are performed on Pythia-70M in a CPU-only environment and use small evaluation slices from WikiText-103 and PG-19. Therefore, the reported speedups should be interpreted as indicative implementation-level evidence under constrained course-project conditions, not as general claims about large-scale GPU inference. Larger models, longer prompts, fused attention kernels, repeated speed trials, and microprofiling of hooks, compression, gather/scatter, and decode time may change the relative benefit of KVPress and GQA-style modifications.

5 Conclusion

The strongest claim supported by these experiments is limited but useful: training-free KVPress can produce speedups in some CPU settings, but the benefit is context- and dataset-dependent. For this CPU-only Pythia-70M setting, KVPress is worth trying when moderate cached-PPL degradation is acceptable, but naive GQA conversion should be avoided unless followed by uptraining.

References

- Biderman, S., Schoelkopf, H., Anthony, Q., Bradley, H., O'Brien, K., Hallahan, E., Khan, M. A., Purohit, S., Prashanth, U. S., Raff, E., Skowron, A., Sutawika, L., and Van Der Wal, O. Pythia: A suite for analyzing large language models across training and scaling. *International Conference on Machine Learning*, 2023.
- Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebron, F., and Sanghai, S. GQA: Training generalized multi-query transformer models from multi-head checkpoints. *Empirical Methods in Natural Language Processing*, 2023.
- Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Re, C. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *Advances in Neural Information Processing Systems*, 2022.
- Xiao, G., Tian, Y., Chen, B., Han, S., and Lewis, M. Efficient streaming language models with attention sinks. arXiv:2309.17453, 2023.
- NVIDIA. KVPress: KV cache compression for efficient LLM inference. <https://github.com/NVIDIA/kvpress>, 2024.
- Merity, S., Xiong, C., Bradbury, J., and Socher, R. Pointer sentinel mixture models. arXiv:1609.07843, 2016.
- Rae, J. W., Potapenko, A., Jayakumar, S. M., and Lillicrap, T. P. Compressive transformers for long-range sequence modelling. *International Conference on Learning Representations*, 2020.

137 A Qualitative Generation Example

138 The following short continuations were generated greedily with
139 scripts/run_qualitative_examples.ps1. The prompt was “Efficient language
140 model inference matters because long prompts require the model to”. Full outputs are saved in
141 outputs/qualitative_examples.md. The example is not used as a formal metric; it is
142 only a sanity check that the GQA degradation is visible in generation behavior.

143 **Baseline.** be a valid and reliable model. The model is a model that is not a valid and reliable
144 model. It is not a model that is valid and reliable.

145 **KVPress-StreamingLLM $r=0.5$.** be used in the analysis of the data. The model is a mixture
146 of two or more parameters, and the model is a mixture of two or more parameters.

147 **GQA-2KV.** the same as the same as the same as the same as the same as the same as the
148 same as the same as the same as the same as